

Deal or No Deal

HW for TAC-368

Your goal is to write a GUI app that let's the user play Deal Or No Deal (once through). (See videos on the Web if you have never seen it.) I am reducing the number of suitcases to 10 so that it does not take so long to play it (and test it). And I'm ditching the 3-at-a-time phase so it is easier to program.

The Rules:

The game starts with 10 suitcases. The suitcases contain the hidden values \$1 5 10 100 1000 5000 10000 100000 500000 1000000, one in each suitcase, randomized so you don't know which suitcase contains which value.

The Player (user) picks one suitcase and holds it, unopened. The Player will get the contents of *this* suitcase at the end unless the Player accepts a deal from the Dealer before then. So what is the Dealer offering? To play the game, the user opens suitcases one at a time. This takes that case out of play, and it also reveals how much money was in it. After each case opening, the Dealer offers to buy the Player's suitcase back for a certain amount of money. The offer is typically for a little less than what the hold-suitcase probably has in it, based on the hidden values in the suitcases left in play.

So what makes this interesting? If the Player accepts an offer, "deal", that's the end. But if the Player rejects an offer with "no deal" (and after every such rejection), the Player must choose to reveal ONE of the remaining hidden-value suitcases. The revealed suitcase is then removed from play. This shrinks the pool of hidden-value suitcases, and thus gives some information about what may be in the hidden suitcase that the Player is holding. If a large value is removed, the Player's case is now among smaller values, so worth less; if a small value is removed, the Player's case has a probability of being worth more. The Player rides this rise and fall, and weighs at each turn the sure thing (the Dealer's offer) vs. the possibility of gain and the risk of loss in the remaining pool of hidden cases, one of which is of course THEIR hold-suitcase.

What to Display

The information about which values have been revealed and which remain hidden should be shown in a table. You should display this as a list in order of value with the revealed ones darkened or something. Technically, this is all that is really needed to play the game, since the Player's suitcase choices are effectively random.

However, in order to give the Player the feeling of some control, we also have the suitcases laid out in some pattern and labeled somehow. When the Player picks one (clicks on it or selects by id or keyboard focus), that suitcase should be opened and its value revealed. That value should be grayed out in the table, and a new offer from the dealer shows up.

The Dealer's offer should be 90% of the expected value (average) of the remaining hidden cases. And you need two big buttons with "DEAL" and "NO DEAL", which the Player uses to end the game or keep going. Oh, and when the game ends, announce the Player's winnings.

Functionality

The program should control the flow of the game. The user should only be able to click "deal" or "no deal" when there is the offer of a deal. The user should only be able to click on a suitcase after clicking "no deal" (except for picking your 'hold' suitcase at the beginning). Your code should enforce these rules. You can keep buttons on the screen and deactivate them, or you can remove them any time they are not in play.

You should also make the game able to run from the keyboard. Use 'd' and 'n' for "deal" and "no deal". The suitcases can be identified by number and picked by hitting a number key.

Submission

Submit your program on github and/or post a zip file to BrightSpace. Do not zip the whole project. I just need the .dart files for this program, and if you use images, put them in the folder that you zip. `deal_yourname` would be a good name for your submission.

Note: You guys have been good about having structure to your program and not just copy+pasting all over the place. Keep that up. If you find yourself copy+pasting a lot of code, stop and think how to do it better.